

About MCP

Architecture overview

Copy page



This overview of the Model Context Protocol (MCP) discusses its scope and core concepts, and provides an example demonstrating each core concept.

Because MCP SDKs abstract away many concerns, most developers will likely find the data layer protocol section to be the most useful. It discusses how MCP servers can provide context to an AI application.

For specific implementation details, please refer to the documentation for your language-specific SDK.

Scope

The Model Context Protocol includes the following projects:

- **MCP Specification**: A specification of MCP that outlines the implementation requirements for clients and servers.
- **MCP SDKs**: SDKs for different programming languages that implement MCP.
- **MCP Development Tools**: Tools for developing MCP servers and clients, including the **MCP Inspector**
- **MCP Reference Server Implementations**: Reference implementations of MCP servers.

MCP focuses solely on the protocol for context exchange—it does not dictate how AI applications use LLMs or manage the provided context.

Concepts of MCP

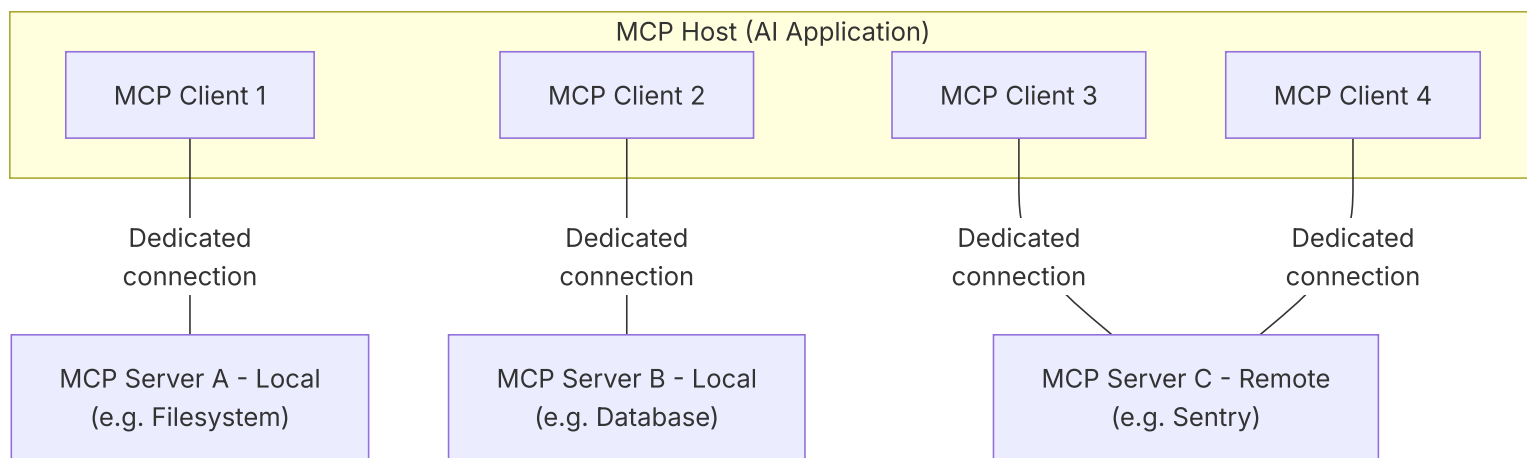
Participants

Local MCP servers that use the STDIO transport typically serve a single MCP client, whereas remote MCP servers that use the Streamable HTTP transport will typically serve many MCP clients.

The key participants in the MCP architecture are:

- **MCP Host:** The AI application that coordinates and manages one or multiple MCP clients
- **MCP Client:** A component that maintains a connection to an MCP server and obtains context from an MCP server for the MCP host to use
- **MCP Server:** A program that provides context to MCP clients

For example: Visual Studio Code acts as an MCP host. When Visual Studio Code establishes a connection to an MCP server, such as the **Sentry MCP server**, the Visual Studio Code runtime instantiates an MCP client object that maintains the connection to the Sentry MCP server. When Visual Studio Code subsequently connects to another MCP server, such as the **local filesystem server**, the Visual Studio Code runtime instantiates an additional MCP client object to maintain this connection.



Note that **MCP server** refers to the program that serves context data, regardless of where it runs. MCP servers can execute locally or remotely. For example, when Claude Desktop launches the **filesystem server**, the server runs locally on the same machine because it uses the STDIO transport. This is commonly referred to as a “local” MCP server. The official **Sentry MCP server** runs on the Sentry platform, and uses the Streamable HTTP transport. This is commonly referred to as a “remote” MCP server.

capability and version discovery, and core primitives, such as tools, resources, prompts and notifications.

- **Transport layer:** Defines the communication mechanisms and channels that enable data exchange between clients and servers, including transport-specific connection establishment, message framing, and authorization.

Conceptually the data layer is the inner layer, while the transport layer is the outer layer.

Data layer

The data layer implements a **JSON-RPC 2.0** based exchange protocol that defines the message structure and semantics. This layer includes:

- **Discovery:** Lets clients query a server's supported protocol versions, capabilities, and identity through the `server/discover` request
- **Server features:** Enables servers to provide core functionality including tools for AI actions, resources for context data, and prompts for interaction templates from and to the client
- **Client features:** Enables servers to elicit input from the user. Sampling is **deprecated** as of protocol version `2026-07-28` .
- **Utility features:** Supports additional capabilities like notifications for real-time updates and progress tracking for long-running operations

Transport layer

The transport layer manages communication channels and authentication between clients and servers. It handles connection establishment, message framing, and secure communication between MCP participants.

MCP supports two transport mechanisms:

- **Stdio transport:** Uses standard input/output streams for direct process communication between local processes on the same machine, providing optimal performance with no network overhead.
- **Streamable HTTP transport:** Uses HTTP POST for client-to-server messages with optional Server-Sent Events for streaming capabilities. This transport enables remote server

MCP 2.0 message format across all transport mechanisms.

Data Layer Protocol

A core part of MCP is defining the schema and semantics between MCP clients and MCP servers. Developers will likely find the data layer — in particular, the set of **primitives** — to be the most interesting part of MCP. It is the part of MCP that defines the ways developers can share context from MCP servers to MCP clients.

MCP uses **JSON-RPC 2.0** as its underlying RPC protocol. Client and servers send requests to each other and respond accordingly. Notifications can be used when no response is required.

Statelessness and discovery

MCP is a stateless protocol. Every request carries the protocol version and the capabilities relevant to that request in its `_meta` field, so the server can process each request on its own. Clients should also identify themselves in the same field unless configured not to. Servers advertise their supported versions and capabilities through the mandatory server/discover request, which clients may send before any other request. Detailed information can be found in the **specification**, and the **example** showcases the per-request metadata and the discovery sequence.

Primitives

MCP primitives are the most important concept within MCP. They define what clients and servers can offer each other. These primitives specify the types of contextual information that can be shared with AI applications and the range of actions that can be performed.

MCP defines three core primitives that *servers* can expose:

- **Tools:** Executable functions that AI applications can invoke to perform actions (e.g., file operations, API calls, database queries)
- **Resources:** Data sources that provide contextual information to AI applications (e.g., file contents, database records, API responses)
- **Prompts:** Reusable templates that help structure interactions with language models (e.g., system prompts, few-shot examples)

As a concrete example, consider an MCP server that provides context about a database. It can expose tools for querying the database, a resource that contains the schema of the database, and a prompt that includes few-shot examples for interacting with the tools.

For more details about server primitives see [server concepts](#).

MCP also defines primitives that *clients* can expose. These primitives allow MCP server authors to build richer interactions.

- **Elicitation:** Allows servers to request additional information from users. This is useful when server authors want to get more information from the user, or ask for confirmation of an action. Servers request user input with the `elicitation/create` method.

Elicitation requests are delivered through the [Multi Round-Trip Requests](#) pattern, explained in the [elicitation overview](#).

Deprecated: The following client primitives are deprecated as of protocol version `2026-07-28`.

- **Sampling:** Allows servers to request language model completions from the client's AI application. This is useful when server authors want access to a language model, but want to stay model-independent and not include a language model SDK in their MCP server. Servers request completions with the `sampling/createMessage` method, also delivered through the Multi Round-Trip Requests pattern. New implementations should integrate directly with LLM provider APIs.
- **Logging:** Enables servers to send log messages to clients for debugging and monitoring purposes. New implementations should log to `stderr` (stdio transport) or use OpenTelemetry.

For more details about client primitives see [client concepts](#).

Besides server and client primitives, the protocol supports optional [extensions](#) that build on the core protocol. For example, the [Tasks extension](#) lets servers return a durable handle for long-running requests, so clients can poll for status and retrieve the result later.

Notifications

(without expecting a response). Change notifications are opt-in: the client opens a long-lived `subscriptions/listen` stream naming the notification types it wants to receive, and the server delivers matching notifications on that stream.

Example

Data Layer

This section provides a step-by-step walkthrough of an MCP client-server interaction, focusing on the data layer protocol. We'll demonstrate discovery, tool operations, and notifications using JSON-RPC 2.0 messages.

1 Discovery

As described in the [**statelessness and discovery**](#) section, every MCP request carries the protocol version and client capabilities in its `_meta` field, and clients should also include their identity there. A client that wants to learn what a server supports before issuing other requests sends a `server/discover` request, which every server must implement. The discovery response is typically cacheable, meaning it can be re-used so the discovery flow does not need to be performed for every request.

```
"method": "server/discover",
"params": {
  "_meta": {
    "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/clientInfo": {
      "name": "example-client",
      "version": "1.0.0"
    },
    "io.modelcontextprotocol/clientCapabilities": {
      "elicitation": {}
    }
  }
}
```

Understanding the Discovery Exchange

The `_meta` fields and the discovery response together serve several purposes:

- Protocol Version Selection:** The `io.modelcontextprotocol/protocolVersion` field declares the version the client is speaking on this request, and `supportedVersions` in the response lists the versions the server accepts. If a server does not support the requested version, it rejects the request with an `UnsupportedProtocolVersionError` listing the versions it does support, and the client retries with a mutually supported version.
- Capability Discovery:** The client declares its capabilities in `io.modelcontextprotocol/clientCapabilities` on every request, and the server returns its own `capabilities` object from `server/discover`. This tells each party which primitives the other can handle (tools, resources, prompts) and whether change notifications are available, so unsupported operations are never attempted.
- Identity Exchange:** The `io.modelcontextprotocol/clientInfo` field in the request's `_meta` and the `io.modelcontextprotocol/serverInfo` field in the result's `_meta` provide identification and versioning information for debugging and compatibility purposes.

In this example, the exchange demonstrates how MCP capabilities are declared:

Server Capabilities:

- `"tools": {"listChanged": true}` - The server supports the tools primitive and can honor a `toolsListChanged` filter in `_subscriptions/listen`. Clients that request this filter receive `notifications/tools/list_changed` when the tool list changes.
- `"resources": {}` - The server also supports the resources primitive (can handle `resources/list` and `resources/read` methods)

Calling `server/discover` is optional. Because every request carries the same `_meta` fields, a client is free to send any request directly and handle a version error if one comes back. Discovery is a convenient way to fetch the server's identity, capabilities, and supported versions in a single request.

How This Works in AI Applications

The AI application's MCP client manager connects to configured servers and stores their discovered capabilities for later use. The application uses this information to determine which servers can provide specific types of functionality (tools, resources, prompts) and whether they support real-time updates. In the Python SDK, discovery happens while the client connects. The results are then available on the client object.

Pseudo-code for AI application discovery

```
# Pseudo Code
async with Client(stdio_client(server_config)) as client:
    if client.server_capabilities.tools:
        app.register_mcp_server(client, supports_tools=True)
    app.set_server_ready(client)
```

Tool Discovery (Primitives)

The client can discover available tools by sending a `tools/list` request. This request is fundamental to MCP's tool discovery mechanism: it allows clients to understand what tools are available on the server before attempting to use them.

```
"method": "tools/list",
"params": {
  "_meta": {
    "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/clientInfo": {
      "name": "example-client",
      "version": "1.0.0"
    },
    "io.modelcontextprotocol/clientCapabilities": {
      "elicitation": {}
    }
  }
}
```

Understanding the Tool Discovery Request

The `tools/list` request requires no parameters beyond the standard `_meta` fields that accompany every MCP request. It also accepts an optional `cursor` parameter for pagination, which the example above omits.

Understanding the Tool Discovery Response

The response contains a `tools` array that provides comprehensive metadata about each available tool. This array-based structure allows servers to expose multiple tools simultaneously while maintaining clear boundaries between different functionalities.

Each tool object in the response includes several key fields:

- **name** : A unique identifier for the tool within the server's namespace. This serves as the primary key for tool execution and should follow a clear naming pattern (e.g., `calculator_arithmetic` rather than just `calculate`)
- **title** : A human-readable display name for the tool that clients can show to users
- **description** : Detailed explanation of what the tool does and when to use it

freshness hint in milliseconds, so this tool list can be cached for five minutes. `cacheScope` indicates who may reuse the response. The specification's [caching utility](#) defines the full rules.

How This Works in AI Applications

The AI application fetches available tools from all connected MCP servers and combines them into a unified tool registry that the language model can access. This allows the LLM to understand what actions it can perform and automatically generates the appropriate tool calls during conversations.

Pseudo-code for AI application tool discovery

```
# Pseudo-code using MCP Python SDK patterns
available_tools = []
for client in app.mcp_clients():
    tools_response = await client.list_tools()
    available_tools.extend(tools_response.tools)
conversation.register_available_tools(available_tools)
```

Clients that federate many servers can use [progressive tool discovery](#) rather than loading every tool upfront.

3

Tool Execution (Primitives)

The client can now execute a tool using the `tools/call` method. This demonstrates how MCP primitives are used in practice: after discovering available tools, the client can invoke them with appropriate arguments.

Understanding the Tool Execution Request

The `tools/call` request follows a structured format that ensures type safety and clear communication between client and server. Note that we're using the proper tool name from the discovery response (`weather_current`) rather than a simplified name:

```
"method": "tools/call",
"params": {
  "name": "weather_current",
  "arguments": {
    "location": "San Francisco",
    "units": "imperial"
  },
  "_meta": {
    "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/clientInfo": {
      "name": "example-client",
      "version": "1.0.0"
    },
    "io.modelcontextprotocol/clientCapabilities": {
      "elicitation": {}
    }
  }
}
```

Key Elements of Tool Execution

The request structure includes several important components:

1. **name** : Must match exactly the tool name from the discovery response (`weather_current`). This ensures the server can correctly identify which tool to execute.
2. **arguments** : Contains the input parameters as defined by the tool's `inputSchema` . In this example:
 - `location` : "San Francisco" (required parameter)
 - `units` : "imperial" (optional parameter, defaults to "metric" if not specified)
3. **_meta** : Carries the standard per-request fields: the protocol version and client capabilities that every MCP request must include, plus the client's identity, which clients should include unless configured not to.

The response demonstrates MCP's flexible content system:

1. **content Array:** Tool responses return an array of content objects, allowing for rich, multi-format responses (text, images, resources, etc.)
2. **Content Types:** Each content object has a `type` field. In this example, `"type": "text"` indicates plain text content, but MCP supports various content types for different use cases.
3. **Structured Output:** The response provides actionable information that the AI application can use as context for language model interactions.

This execution pattern allows AI applications to dynamically invoke server functionality and receive structured responses that can be integrated into conversations with language models.

How This Works in AI Applications

When the language model decides to use a tool during a conversation, the AI application intercepts the tool call, routes it to the appropriate MCP server, executes it, and returns the results back to the LLM as part of the conversation flow. This enables the LLM to access real-time data and perform actions in the external world.

```
# Pseudo-code for AI application tool execution
async def handle_tool_call(conversation, tool_name, arguments):
    client = app.find_mcp_client_for_tool(tool_name)
    result = await client.call_tool(tool_name, arguments)
    conversation.add_tool_result(result.content)
```

4

Real-time Updates (Notifications)

MCP supports real-time notifications that enable servers to inform clients about changes without being polled for them. This demonstrates the notification system, a key feature that keeps clients synchronized and responsive.

Subscribing to Changes



Listen Request

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "method": "subscriptions/listen",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo": {
        "name": "example-client",
        "version": "1.0.0"
      },
      "io.modelcontextprotocol/clientCapabilities": {
        "elicitation": {}
      }
    },
    "notifications": {
      "toolsListChanged": true
    }
  }
}
```

Every client request carries the `io.modelcontextprotocol/protocolVersion` and `io.modelcontextprotocol/clientCapabilities` fields in `_meta`, and normally `io.modelcontextprotocol/clientInfo` as well, so the server can identify the client without relying on connection state.

The server acknowledges the subscription with `notifications/subscriptions/acknowledged`, which is the first message carrying that subscription's ID in `_meta` (the server sends no other notification for that subscription before it). Its `notifications` field reflects the subset of the requested filter the server agreed to honor, with unsupported notification types omitted:

```
"params": {
  "_meta": {
    "io.modelcontextprotocol/subscriptionId": 4
  },
  "notifications": {
    "toolsListChanged": true
  }
}
```

Understanding Tool List Change Notifications

After the acknowledgment, when the server's available tools change (for example, when new functionality becomes available, existing tools are modified, or tools become temporarily unavailable), the server delivers a notification on that stream:

Notification

```
{
  "jsonrpc": "2.0",
  "method": "notifications/tools/list_changed",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/subscriptionId": 4
    }
  }
}
```

Key Features of MCP Notifications

1. **No Response Required:** Notice there's no `id` field in the notification. This follows JSON-RPC 2.0 notification semantics where no response is expected or sent.
2. **Opt-In Based:** This notification is only sent to clients that requested `"toolsListChanged": true` in their `subscriptions/listen` filter, and it is only available from servers that declared `"listChanged": true` in their tools capability (as shown in Step 1).

4. **Event-Driven:** The server decides when to send notifications based on internal state changes, making MCP connections dynamic and responsive.
5. **Best Effort:** There are no guarantees that every notification will be sent or received, particularly across transport reconnects. Clients should also rely on polling to preserve freshness of results.

Client Response to Notifications

Upon receiving this notification, the client typically reacts by requesting the updated tool list. This creates a refresh cycle that keeps the client's understanding of available tools current:

Request

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "method": "tools/list",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientInfo": {
        "name": "example-client",
        "version": "1.0.0"
      },
      "io.modelcontextprotocol/clientCapabilities": {
        "elicitation": {}
      }
    }
  }
}
```

Why Notifications Matter

This notification system is crucial for several reasons:

capabilities

4. **Real-time Collaboration:** Enables responsive AI applications that can adapt to changing contexts

This notification pattern extends beyond tools to other MCP primitives, enabling comprehensive real-time synchronization between clients and servers.

How This Works in AI Applications

The AI application keeps a notification stream open for the changes it cares about. When one arrives, it immediately refreshes its tool registry and updates the LLM's available capabilities. This ensures that ongoing conversations always have access to the most current set of tools, and the LLM can dynamically adapt to new functionality as it becomes available.

```
# Pseudo-code for AI application notification handling
async def follow_tool_changes(client):
    async with client.listen(tools_list_changed=True) as sub:
        async for _event in sub:
            tools_response = await client.list_tools()
            app.update_available_tools(client, tools_response.tools)
            if app.conversation.is_active():
                app.conversation.notify_llm_of_new_capabilities()
```

Was this page helpful?

 Yes

 No

